

PHASE 3

Backend Services & Asynchronous Processing

Task-Based Training Plan for ASP.NET Core Developers

Redis • RabbitMQ • File Operations • Background Workers • Service Communication

Program Detail	Scope
System	Continue the existing Trainee Management System
Primary stack	C#, ASP.NET Core Web API, MySQL, Redis, RabbitMQ
Environment	Docker and Docker Compose are prerequisites and are not covered in this phase
Recommended duration	7 working days
Delivery model	Daily task completion, Git commits, mentor review, and final integrated demo

Important: Continue from the Phase 2 backend project. Do not create a replacement application.

1. Training Objective

Phase 3 moves the project beyond conventional CRUD APIs. Trainees will implement file handling, distributed caching, asynchronous messaging, background processing, and service-to-service communication while preserving the security, validation, logging, and database practices established in earlier phases.

- Upload, download, validate, and securely store trainee submission files.
- Use Redis as a distributed cache with an explicit cache-aside and invalidation strategy.
- Publish and consume durable work messages through RabbitMQ.
- Execute long-running operations through a separate background worker.
- Call another internal HTTP service through `IHttpClientFactory` with timeout and resilience controls.
- Trace an operation across API, database, Redis, RabbitMQ, and worker logs using a correlation ID.
- Handle retries, duplicate delivery, failed messages, and partial failures safely.

2. System Scenario

Extend the Trainee Management System with an asynchronous submission-processing workflow:

1. A trainee uploads a submission file against an existing task assignment.
2. The API validates the request, stores the physical file through a storage abstraction, and saves file metadata in MySQL.
3. The API publishes a submission-processing message to RabbitMQ and returns an accepted response with a tracking identifier.
4. A background worker consumes the message, retrieves the file, performs simulated processing, and updates the submission status.
5. Frequently requested submission and assignment information is cached in Redis.
6. The worker calls an internal service to retrieve additional trainee or assignment information.
7. Logs and status APIs allow the full workflow to be observed and diagnosed.

3. Required Project Components

Component	Responsibility
TraineeManagement.Api	Upload/download APIs, metadata APIs, cache access, RabbitMQ publishing, job-status APIs
SubmissionProcessor.Worker	RabbitMQ consumer, file processing, status updates, retry and failure handling
TrainingDirectory.Api	Small internal service that exposes trainee/assignment summary data for service communication
MySQL	Authoritative application and job metadata store
Redis	Distributed cache for selected read models
RabbitMQ	Work queue for asynchronous submission processing
Docker Compose	Runs MySQL, Redis, RabbitMQ, APIs, and worker locally

4. Recommended Daily Plan

Day	Primary Focus	Daily Outcome
1	Secure file operations	Upload, download, metadata, validation, and storage abstraction
2	Redis caching	Cache-aside reads, TTL, invalidation, and graceful cache failure
3	RabbitMQ producer/consumer	Durable queue, publish, consume, acknowledgements, and persistence
4	Background processing	Worker service, job lifecycle, idempotency, retry, and failed-message handling
5	Inter-service communication	Internal API client, IHttpConnectionFactory, timeout, retry, and cancellation
6	Integration and observability	Correlation IDs, health checks, structured logs, and end-to-end workflow
7	Hardening and final demo	Security review, failure tests, documentation, and integrated demonstration

5. Day 1 — Secure File Operations

Objective: Add secure file upload and download support without coupling business logic to local disk storage.

Task 3.1: Create a file-storage abstraction

- Create `IFileStorageService` with `SaveAsync`, `OpenReadAsync`, `ExistsAsync`, and `DeleteAsync` operations.
- Implement `LocalFileStorageService` for the training environment.
- Read the storage root from configuration; do not hardcode machine-specific paths.
- Generate server-side storage names. Never use the uploaded file name as the physical file name.

Deliverable: Storage interface, local implementation, configuration, and dependency-injection registration.

[Microsoft Learn: Upload files in ASP.NET Core](#)

Task 3.2: Create file metadata and upload API

- Create `SubmissionFile` metadata linked to `TaskAssignment` or `Submission`.
- Store original file name, generated storage name, content type, size, checksum, uploaded-by user, and timestamps in MySQL.
- Create `POST /api/submissions/{submissionId}/files` using multipart/form-data.
- Reject empty files, files above the configured limit, disallowed extensions, and invalid ownership or authorization.

Expected response: Return 201 Created with file identifier and metadata. Do not expose the physical storage path.

Task 3.3: Create download and delete APIs

- Create `GET /api/submission-files/{id}/download`.
- Authorize access before opening the file.
- Return 404 when metadata or physical content does not exist.
- Create `DELETE /api/submission-files/{id}` and keep database/file operations consistent.

Task 3.4: Apply file-security controls

- Validate extension and declared content type; do not rely on the client-provided file name.
- Prevent path traversal by never combining a client file name directly with a storage path.
- Configure upload-size limits and return a clear 413 or validation response.
- Log metadata and result, but never log file contents or secrets.

[OWASP: File Upload Cheat Sheet](#)

Day 1 submission: Working upload/download/delete flow, migration, Swagger/Postman evidence, and at least five negative validation tests.

6. Day 2 — Redis Distributed Caching

Objective: Improve read performance without making Redis the source of truth.

Task 3.5: Run and configure Redis

- Add Redis to the existing Docker Compose environment.
- Configure ASP.NET Core distributed caching using `Microsoft.Extensions.Caching.StackExchangeRedis`.
- Store the Redis connection details in configuration or environment variables.
- Verify connectivity without exposing credentials in logs.

[Microsoft Learn: Distributed caching in ASP.NET Core](#)

[Redis documentation: .NET clients](#)

Task 3.6: Implement cache-aside for read APIs

- Cache selected trainee, task-assignment, and submission-summary responses.
- On a cache miss, read from MySQL, serialize the response, store it with a TTL, and return it.
- Use a predictable key convention such as `trainee:{id}` or `submission-summary:{id}`.
- Do not cache password hashes, JWTs, uploaded content, or authorization decisions.

Task 3.7: Implement cache invalidation

- Remove or refresh related keys after create, update, delete, submission upload, and processing-status changes.
- Document which write operation invalidates each key.
- Demonstrate that an update is reflected immediately and stale data is not returned.

Task 3.8: Handle Redis failure safely

- Treat cache failure as a performance degradation, not as loss of the core API.
- Log cache errors and fall back to MySQL where appropriate.
- Do not hide database failures by serving indefinitely stale cache data.

Day 2 submission: Redis configuration, cache service, hit/miss demonstration, invalidation tests, and evidence of database fallback when Redis is unavailable.

7. Day 3 — RabbitMQ Messaging

Objective: Move submission processing out of the request path by publishing durable work messages.

Task 3.9: Run and configure RabbitMQ

- Add RabbitMQ with the management plugin to Docker Compose.
- Read host, port, virtual host, username, and password from configuration.
- Use a durable queue named `submission-processing`.
- Use persistent messages for work that must survive a broker restart.

[RabbitMQ official .NET tutorial: Work Queues](#)

[RabbitMQ official .NET client guide](#)

Task 3.10: Define a versioned message contract

- Create `SubmissionProcessingRequested` with `MessageId`, `CorrelationId`, `SubmissionId`, `FileId`, `RequestedAt`, and `ContractVersion`.
- Do not send passwords, tokens, physical file paths, or unnecessary user data.
- Keep the contract independent of EF Core entity classes.

Task 3.11: Publish messages after a valid submission

- Publish only after database validation and file metadata persistence succeeds.
- Return 202 Accepted with a tracking identifier for asynchronous work.
- Log the message ID, correlation ID, submission ID, and publish result.
- Handle broker unavailability and avoid falsely reporting that work was queued.

Task 3.12: Create a basic consumer

- Create a separate Worker Service project.
- Consume one message at a time initially using an appropriate prefetch value.
- Acknowledge only after processing and status persistence succeed.
- Negatively acknowledge or route failed work according to the retry design.

Day 3 submission: RabbitMQ setup, message contract, publisher, consumer skeleton, management-console evidence, and restart/persistence test.

8. Day 4 — Background Worker and Reliable Processing

Objective: Implement an observable and repeatable asynchronous job lifecycle.

Task 3.13: Create the submission processor

- Implement SubmissionProcessor.Worker using the .NET Worker Service model.
- Load scoped services correctly inside the long-running worker.
- Simulate processing by calculating a checksum, extracting safe metadata, or producing a generated result file.
- Update status through Queued, Processing, Completed, or Failed states.

[Microsoft Learn: Worker services in .NET](#)

[Microsoft Learn: Background tasks with hosted services](#)

Task 3.14: Add job tracking

- Create ProcessingJob with status, attempts, error summary, started/completed timestamps, and correlation ID.
- Create GET /api/processing-jobs/{id}.
- Ensure the API reports the authoritative database state rather than only broker state.

Task 3.15: Make the consumer idempotent

- Detect an already-completed MessageId or FileId and do not process it twice.
- Store enough processing history to handle redelivery safely.
- Demonstrate duplicate-message handling without duplicate database updates or output files.

Task 3.16: Add retry and failed-message handling

- Classify transient and permanent failures.
- Limit retry attempts and avoid an endless requeue loop.
- Route exhausted messages to a failed/dead-letter queue and persist a failed job state.
- Include sufficient logs to diagnose the failure without exposing sensitive data.

[RabbitMQ documentation: Dead Letter Exchanges](#)

Day 4 submission: End-to-end asynchronous processing, status API, duplicate-delivery test, retry demonstration, and failed-message evidence.

9. Day 5 — Inter-Service Communication and Resilience

Objective: Call another internal HTTP service predictably and safely.

Task 3.17: Create a small internal service

- Create TrainingDirectory.Api as a separate project/container.
- Expose a read-only endpoint that returns a trainee or assignment processing profile.
- Keep the scope small; this service exists to practise service-to-service communication.

Task 3.18: Create a typed HTTP client

- Use IHttpClientFactory; do not instantiate a new unmanaged HttpClient for every request.
- Configure BaseAddress, timeout, and required headers centrally.
- Use typed request/response contracts and handle non-success responses explicitly.
- Propagate correlation ID and CancellationToken.

[Microsoft Learn: Use IHttpClientFactory](#)

[Microsoft Learn: Make HTTP requests using IHttpClientFactory](#)

Task 3.19: Add resilience controls

- Apply a finite timeout.
- Retry only operations that are safe to retry and only for transient failures.
- Add a circuit-breaker or standard resilience handler.
- Define fallback behaviour when the internal service is unavailable.
- Do not retry validation errors, authentication failures, or every HTTP status indiscriminately.

[Microsoft Learn: Build resilient HTTP apps](#)

Task 3.20: Compare synchronous and asynchronous communication

- Document why the directory lookup is synchronous while file processing is asynchronous.
- Identify latency, availability, coupling, and consistency trade-offs.
- Explain what happens when each dependency is unavailable.

Day 5 submission: Internal service, typed client, timeout/retry/circuit-breaker evidence, correlation propagation, and a short design note.

10. Day 6 — Observability, Health, and End-to-End Integration

Objective: Make the distributed workflow diagnosable and demonstrate the complete path.

Task 3.21: Add structured and correlated logging

- Use structured log properties instead of concatenated text.
- Carry CorrelationId from the incoming API request into the database job, RabbitMQ message, worker, and internal HTTP request.
- Log state transitions, cache hit/miss, publish/consume result, retry attempt, and dependency failure.
- Never log passwords, JWTs, connection strings, or file contents.

[Microsoft Learn: Logging in C# and .NET](#)

Task 3.22: Add health checks

- Expose liveness and readiness endpoints.
- Check MySQL, Redis, RabbitMQ, and the internal HTTP service where practical.
- Do not expose sensitive dependency details in public health responses.

[Microsoft Learn: Health checks in ASP.NET Core](#)

Task 3.23: Complete Docker Compose integration

- Run API, worker, internal service, MySQL, Redis, and RabbitMQ as one local stack.
- Use service names for container-to-container connections rather than localhost.
- Add dependency health checks and suitable restart policies.
- Keep credentials in environment configuration rather than committed source files.

[Docker Docs: Compose](#)

Task 3.24: Execute the end-to-end workflow

8. Authenticate and obtain a JWT.
9. Upload a valid submission file.
10. Receive 202 Accepted and a tracking ID.
11. Confirm a RabbitMQ message is published and consumed.
12. Observe the worker calling the internal service.
13. Verify job status becomes Completed and relevant cache is invalidated.
14. Download the source or generated output file with authorization.
15. Trace the entire operation using one correlation ID.

Day 6 submission: Running Docker Compose stack, health results, correlated logs, and complete Postman/Swagger workflow evidence.

11. Day 7 — Hardening, Testing, and Final Demo

Objective: Validate behaviour under failure and prepare a professional submission.

Task 3.25: Run mandatory failure tests

Failure Scenario	Expected Behaviour
Invalid or oversized upload	Request rejected with a clear 4xx response; no metadata or file left behind
Redis unavailable	Read API falls back to MySQL and logs the cache failure

Failure Scenario	Expected Behaviour
RabbitMQ unavailable	API does not falsely claim that processing was queued
Duplicate message	Worker identifies the duplicate and avoids repeated side effects
Internal service timeout	Timeout/resilience policy applies and job records the defined outcome
Worker restart during processing	Unacknowledged work is redelivered and handled idempotently
Permanent processing error	Message reaches failed/dead-letter handling and job becomes Failed

Task 3.26: Complete security review

- Authorization is enforced for upload, download, delete, job status, and administrative operations.
- File names and paths cannot be controlled by the client.
- RabbitMQ, Redis, MySQL, and JWT credentials are not committed to source control.
- Message contracts and logs do not expose secrets or unnecessary personal data.
- Input validation, exception middleware, and HTTP status codes remain consistent with Phase 2.

[OWASP API Security Top 10](#)

Task 3.27: Prepare documentation and final demo

- Update README with architecture, configuration, Docker Compose start-up, migrations, endpoints, and test credentials.
- Add a workflow diagram covering API, MySQL, Redis, RabbitMQ, worker, file storage, and internal service.
- Include design decisions: cache keys/TTL, queue settings, retries, idempotency, and failure behaviour.
- Prepare a concise live demonstration using a clean environment.

Day 7 submission: Final repositories, Docker Compose file, migrations, Postman collection, test evidence, architecture diagram, README, and live demo.

12. Minimum API Scope

Method	Endpoint	Purpose
POST	/api/submissions/{submissionId}/files	Upload and queue a submission file
GET	/api/submission-files/{id}	Return file metadata
GET	/api/submission-files/{id}/download	Authorize and download a file
DELETE	/api/submission-files/{id}	Delete metadata and physical file
GET	/api/processing-jobs/{id}	Return asynchronous processing status
POST	/api/processing-jobs/{id}/retry	Optional controlled retry for an authorized user
GET	/api/submissions/{id}/summary	Cached submission summary
GET	/health/live	Liveness endpoint
GET	/health/ready	Readiness endpoint

13. Mandatory Engineering Expectations

Area	Expectation
Architecture	Controllers remain thin; use services and dependency injection; external infrastructure is abstracted behind interfaces.
Database	MySQL remains the source of truth. Redis and RabbitMQ do not replace authoritative application state.
Async code	Use asynchronous I/O and propagate CancellationToken where supported.
Caching	Explicit cache-aside, TTL, key convention, invalidation, and fallback are documented and tested.
Messaging	Durable work queue, persistent messages, manual acknowledgement, finite retry, and failed-message handling.
Reliability	Consumers are idempotent and safe under redelivery.
Security	Authorization, secure file handling, secret management, validation, and safe logging are mandatory.
Observability	Structured logs, correlation IDs, job status, and dependency health provide traceability.
Source control	Small, meaningful daily commits; no secrets, generated uploads, or local data directories committed.

14. Final Submission Checklist

Item	Required Evidence
Source code	Updated API, worker service, and internal service projects
Infrastructure	Docker Compose configuration for MySQL, Redis, RabbitMQ, APIs, and worker
Database	EF Core migrations and setup instructions
File operations	Positive and negative upload/download/delete tests
Redis	Cache hit, miss, invalidation, expiry, and outage evidence
RabbitMQ	Published/consumed message, acknowledgement, retry, duplicate, and failed-message evidence
Background jobs	Queued-to-completed and queued-to-failed status flows
Service communication	Timeout, retry/circuit behaviour, and correlation propagation
Testing	Postman collection or equivalent repeatable API tests
Documentation	README, architecture diagram, configuration guide, known limitations, and design decisions

15. Scope Boundaries

The following topics are intentionally outside the mandatory scope for this phase:

- Kafka, Kubernetes, service mesh, event sourcing, CQRS, or distributed transactions.
- Cloud object storage implementation such as Amazon S3 or Azure Blob Storage.
- Redis clustering or RabbitMQ clustering/high-availability configuration.
- Complex microservice decomposition or production-grade identity-provider integration.
- React frontend implementation; frontend work remains a separate phase.

17. Official Reference Index

[ASP.NET Core file uploads](#)

[ASP.NET Core distributed caching](#)

[Redis .NET clients](#)

[RabbitMQ .NET work queues tutorial](#)

[RabbitMQ .NET client guide](#)

[RabbitMQ dead-letter exchanges](#)

[.NET Worker Services](#)

[ASP.NET Core hosted services](#)

[IHttpClientFactory](#)

[.NET HTTP resilience](#)

[.NET logging](#)

[ASP.NET Core health checks](#)

[Docker Compose](#)

[OWASP File Upload Cheat Sheet](#)

[OWASP API Security Top 10](#)

Focus on understanding and demonstrating the behaviour. Avoid copying an implementation that you cannot explain.